

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>State-Driven To-Do App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="app-container">
    <header>
      <h1>Task Manager</h1>
    </header>

    <!-- Create Feature -->
    <form id="todo-form">
      <input type="text" id="todo-input" placeholder="What needs to be done?" required
autocomplete="off">
      <button type="submit" id="add-btn">Add Task</button>
    </form>

    <!-- Filter Feature -->
    <div class="filter-container">
      <button class="filter-btn active" data-filter="all">All</button>
      <button class="filter-btn" data-filter="active">Active</button>
      <button class="filter-btn" data-filter="completed">Completed</button>
    </div>

    <!-- Read, Update, Delete Feature -->
    <ul id="todo-list"></ul>

    <!-- App Statistics -->
    <footer class="app-footer">
      <span id="items-left">0 items left</span>
      <button id="clear-completed" class="clear-btn">Clear Completed</button>
    </footer>
  </div>

  <script src="app.js"></script>
</body>
</html>
:root {
  --bg-color: #f5f5f5;
  --card-bg: #ffffff;

```

```
--text-main: #333333;  
--text-muted: #777777;  
--primary: #4a90e2;  
--danger: #e04b4b;  
--border: #e6e6e6;  
}
```

```
body {  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial,  
  sans-serif;  
  background-color: var(--bg-color);  
  color: var(--text-main);  
  margin: 0;  
  padding: 40px 20px;  
  display: flex;  
  justify-content: center;  
}
```

```
.app-container {  
  background: var(--card-bg);  
  width: 100%;  
  max-width: 550px;  
  border-radius: 8px;  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);  
  padding: 20px;  
}
```

```
header h1 {  
  text-align: center;  
  margin-top: 0;  
  color: var(--primary);  
}
```

```
form {  
  display: flex;  
  gap: 10px;  
  margin-bottom: 20px;  
}
```

```
input[type="text"] {  
  flex: 1;  
  padding: 10px 12px;  
  font-size: 16px;  
  border: 1px solid var(--border);  
}
```

```
border-radius: 4px;  
outline: none;  
}
```

```
input[type="text"]:focus {  
  border-color: var(--primary);  
}
```

```
button {  
  cursor: pointer;  
  border: none;  
  border-radius: 4px;  
  font-weight: 600;  
}
```

```
#add-btn {  
  background-color: var(--primary);  
  color: white;  
  padding: 0 16px;  
}
```

```
.filter-container {  
  display: flex;  
  gap: 8px;  
  margin-bottom: 15px;  
}
```

```
.filter-btn {  
  background: none;  
  color: var(--text-muted);  
  padding: 6px 12px;  
  border: 1px solid transparent;  
}
```

```
.filter-btn:hover {  
  border-color: var(--border);  
}
```

```
.filter-btn.active {  
  border-color: var(--primary);  
  color: var(--primary);  
}
```

```
#todo-list {
```

```
list-style: none;
padding: 0;
margin: 0;
border-top: 1px solid var(--border);
}

.todo-item {
display: flex;
align-items: center;
padding: 12px 0;
border-bottom: 1px solid var(--border);
gap: 10px;
}

.todo-item.completed .todo-text {
text-decoration: line-through;
color: var(--text-muted);
}

.todo-item.editing .todo-text {
display: none;
}

.edit-input {
display: none;
flex: 1;
padding: 6px;
font-size: 16px;
}

.todo-item.editing .edit-input {
display: block;
}

.todo-text {
flex: 1;
word-break: break-word;
cursor: pointer;
}

.delete-btn {
background: none;
color: var(--danger);
padding: 4px 8px;
```

```
}
```

```
.app-footer {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-top: 15px;  
  font-size: 14px;  
  color: var(--text-muted);  
}
```

```
.clear-btn {  
  background: none;  
  color: var(--text-muted);  
  font-size: 14px;  
}
```

```
.clear-btn:hover {  
  color: var(--danger);  
  text-decoration: underline;  
}
```

```
// --- STATE MANAGEMENT ---
```

```
let todos = JSON.parse(localStorage.getItem('todos')) || [];  
let currentFilter = 'all';
```

```
// --- DOM ELEMENTS ---
```

```
const todoForm = document.getElementById('todo-form');  
const todoInput = document.getElementById('todo-input');  
const todoList = document.getElementById('todo-list');  
const filterButtons = document.querySelectorAll('.filter-btn');  
const itemsLeftCounter = document.getElementById('items-left');  
const clearCompletedBtn = document.getElementById('clear-completed');
```

```
// --- CRUD OPERATIONS & CORE LOGIC ---
```

```
// Save to LocalStorage
```

```
function saveState() {  
  localStorage.setItem('todos', JSON.stringify(todos));  
  render();  
}
```

```
// CREATE: Add new item
```

```
function addTodo(text) {  
  const newTodo = {
```

```

    id: Date.now().toString(),
    text: text.trim(),
    completed: false
  };
  todos.push(newTodo);
  saveState();
}

// UPDATE: Toggle status
function toggleTodo(id) {
  todos = todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  );
  saveState();
}

// UPDATE: Edit text
function updateTodoText(id, newText) {
  if (newText.trim() === "") {
    deleteTodo(id); // Delete if text cleared
    return;
  }
  todos = todos.map(todo =>
    todo.id === id ? { ...todo, text: newText.trim() } : todo
  );
  saveState();
}

// DELETE: Remove item
function deleteTodo(id) {
  todos = todos.filter(todo => todo.id !== id);
  saveState();
}

// DELETE: Clear all completed items
function clearCompleted() {
  todos = todos.filter(todo => !todo.completed);
  saveState();
}

// --- RENDER PIPELINE ---

function render() {
  // 1. Filter elements based on UI state

```

```

const filteredTodos = todos.filter(todo => {
  if (currentFilter === 'active') return !todo.completed;
  if (currentFilter === 'completed') return todo.completed;
  return true;
});

// 2. Clear out container
todoList.innerHTML = "";

// 3. Dynamic DOM Elements Creation
filteredTodos.forEach(todo => {
  const li = document.createElement('li');
  li.className = `todo-item ${todo.completed ? 'completed' : ''}`;
  li.dataset.id = todo.id;

  li.innerHTML = `
    <input type="checkbox" class="toggle-checkbox" ${todo.completed ? 'checked' : ''}>
    <span class="todo-text">${escapeHTML(todo.text)}</span>
    <input type="text" class="edit-input" value="${escapeHTML(todo.text)}">
    <button class="delete-btn">Delete</button>
  `;

  todoList.appendChild(li);
});

// 4. Update Footer Metrics
const activeCount = todos.filter(todo => !todo.completed).length;
itemsLeftCounter.textContent = `${activeCount} item${activeCount !== 1 ? 's' : ''} left`;
}

// Helper to prevent XSS attacks
function escapeHTML(str) {
  return str.replace(/&<>"/g,
    tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '":': '&#39;', '":': '&quot;' }[tag] || tag)
  );
}

// --- EVENT HANDLERS & DELEGATION ---

// Handle form submission (Create)
todoForm.addEventListener('submit', (e) => {
  e.preventDefault();
  if (todoInput.value.trim()) {
    addTodo(todoInput.value);
  }
});

```

```

    todoInput.value = "";
  }
});

// Event Delegation for To-Do item actions (Update & Delete)
todoList.addEventListener('click', (e) => {
  const target = e.target;
  const todoItem = target.closest('.todo-item');
  if (!todoItem) return;
  const id = todoItem.dataset.id;

  // Toggle Checkbox
  if (target.classList.contains('toggle-checkbox')) {
    toggleTodo(id);
  }

  // Trigger Delete Button
  if (target.classList.contains('delete-btn')) {
    deleteTodo(id);
  }
});

// Event Delegation for Inline Editing (Double Click to edit)
todoList.addEventListener('dblclick', (e) => {
  const target = e.target;
  const todoItem = target.closest('.todo-item');

  if (target.classList.contains('todo-text') && todoItem) {
    todoItem.classList.add('editing');
    const editInput = todoItem.querySelector('.edit-input');
    editInput.focus();
    // Move cursor to the end of text
    const len = editInput.value.length;
    editInput.setSelectionRange(len, len);
  }
});

// Handle edit confirmation and blur events
todoList.addEventListener('keydown', (e) => {
  const target = e.target;
  if (target.classList.contains('edit-input') && (e.key === 'Enter' || e.key === 'Escape')) {
    const todoItem = target.closest('.todo-item');
    const id = todoItem.dataset.id;
  }
});

```



```

    if (e.key === 'Enter') {
      updateTodoText(id, target.value);
    } else if (e.key === 'Escape') {
      render(); // Reset view without saving changes
    }
  }
});

```

```

todoList.addEventListener('focusout', (e) => {
  const target = e.target;
  if (target.classList.contains('edit-input')) {
    const todoItem = target.closest('.todo-item');
    if (todoItem && todoItem.classList.contains('editing')) {
      const id = todoItem.dataset.id;
      updateTodoText(id, target.value);
    }
  }
});

```

```

// Filter selection event
document.querySelector('.filter-container').addEventListener('click', (e) => {
  if (e.target.classList.contains('filter-btn')) {
    filterButtons.forEach(btn => btn.classList.remove('active'));
    e.target.classList.add('active');
    currentFilter = e.target.dataset.filter;
    render();
  }
});

```

```

// Clear completed items button
clearCompletedBtn.addEventListener('click', clearCompleted);

```

```

// --- INITIAL INITIALIZATION ---
render();

```